Homelab Agent-Cluster — Project Plan

Owner: Borna · Last updated: July 18, 2026 · Linux comfort: 3/5

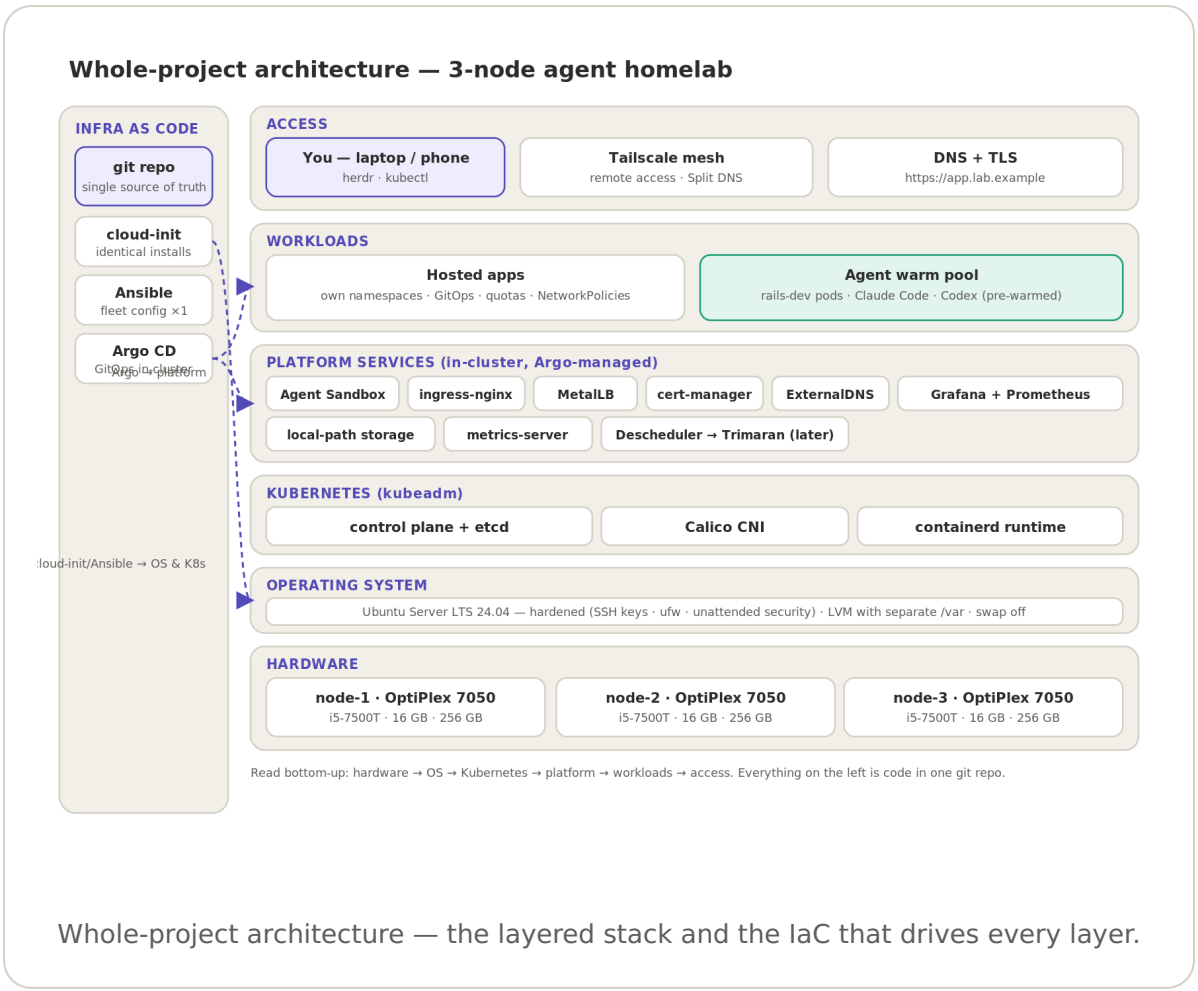
A 3-node Kubernetes homelab that hosts your own apps (GitOps) **and** runs AI coding agents in pre-warmed, disposable environments, all viewable through herdr.

On this page: Architecture · Goals · Definition of done · Decisions · Principles · Build plan · Repo · Day-2 ops · Risks · Glossary

Architecture

1 · Whole project — the full stack

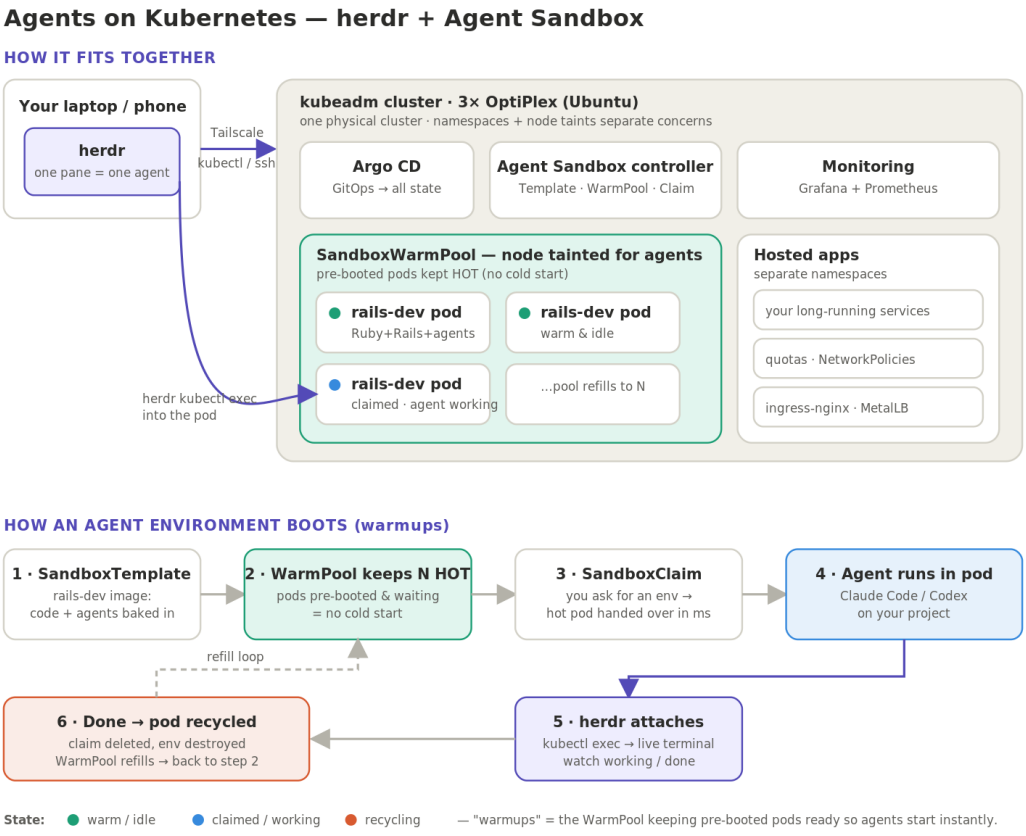
Read bottom-up: **hardware → OS → Kubernetes → platform services → workloads → access**. Everything on the left rail is code in one git repo — cloud-init and Ansible build the OS/Kubernetes layers, Argo CD manages everything inside the cluster.



Whole-project architecture — the layered stack and the IaC that drives every layer.

2 · How agents run — herdr + Agent Sandbox + warm pools

You issue a **SandboxClaim**, get a pre-booted pod instantly, and herdr `kubectl exec`s into it to run Claude Code or Codex — watching each agent go working → done. When finished, the pod is recycled and the pool refills. The **warmups** are the `SandboxWarmPool` keeping pods hot so there's no cold start.



Agent execution — how herdr, Agent Sandbox and Kubernetes fit together, and the warm-pool lifecycle.

Goals

- **Learn Kubernetes properly** — the real, upstream way (kubeadm), not a black box.
- **Run AI coding agents** (Claude Code, Codex; PI later) in isolated, disposable pods, pre-warmed and preloaded with a project (example: a modern Ruby on Rails app).
- **Host my own apps** on the same cluster, managed by GitOps.
- **Everything as code** — nodes and cluster reproducible from a git repo; never configure a machine by hand 3×.

- **Reachable & observable** — access from anywhere over Tailscale, apps by hostname over HTTPS, Grafana dashboards for health.

Definition of done

- Three identical Ubuntu nodes stood up from **one** autoinstall + Ansible run.
- A working **kubeadm** cluster with CNI, storage, ingress, and a bare-metal load balancer.
- **Argo CD** syncing all cluster state from a git repo.
- Apps reachable by **hostname over HTTPS** (internal DNS + cert-manager).
- **Grafana** showing live node/pod metrics; a test alert fires.
- A **rails-dev** sandbox image; **Agent Sandbox** warm pool hands over a ready pod in ~ms.
- **herdr** driving 2 agents in 2 warm pods at once.
- Cluster reachable from outside the house over **Tailscale**.
- At least one **hosted app** deployed via GitOps in its own namespace.
- An **etcd snapshot** taken and a restore rehearsed once.

Decisions at a glance

LAYER	DECISION	WHY (SHORT)
Hardware	3× OptiPlex 7050 (i5-7500T, 16 GB, 256 GB)	Enough for a learning cluster; SSD is the limit
Node OS	**Ubuntu Server LTS 24.04**	5-yr support, no SELinux friction, most tutorials
Kubernetes	**kubeadm** (upstream/vanilla)	Learn the real thing; start 1 CP + 2 workers → 3 CP + kube-vip
Runtime / CNI	containerd · Calico	Standard, beginner-friendly
Storage	local-path (→ Longhorn later)	Simple start; watch SSD space
Ingress / LB	ingress-nginx + **MetalLB**	Bare-metal LoadBalancer + ingress
Hostname access	Local DNS (AdGuard/dnsmasq wildcard) → ingress IP; ExternalDNS + cert-manager; Tailscale Split DNS	Reach apps by name over HTTPS (the "internal CDN" ask — really DNS+ingress)
IaC	cloud-init/autoinstall → **Ansible** → **Argo CD**	Identical installs, fleet config, GitOps
Secrets	kubectl Secrets → **SOPS / Sealed Secrets**	Safe to keep in git
Topology	**One physical cluster** (namespaces + taints + quotas + NetworkPolicies)	3 nodes can't afford a physical split; vCluster later if needed
Disposable envs	**Agent Sandbox** (Template / WarmPool / Claim)	Pre-warmed, isolated agent pods
Agents / view	Claude Code + Codex (PI later) · **herdr**	Baked into image; multi-agent dashboard
Monitoring	**kube-prometheus-stack** (Grafana), retention capped	Standard; sized for small SSDs

LAYER	DECISION	WHY (SHORT)
Scheduling	requests + metrics-server + topology spread → Descheduler → Trimaran	Distribute by load (real-usage later)
Remote access	**Tailscale** mesh	No bridge box needed
Decided out	**Talos migration — not doing it**	Stay on Ubuntu + kubeadm; git makes it reproducible anyway
Parked (optional)	Hetzner box, vCluster, Loki, Trimaran	Add only if a real need appears

Guiding principles

- **Everything as code, in one git repo.** If it's not in git, it doesn't exist.
- **Learn first, automate second.** Do a step by hand once, then capture it in Ansible/Argo.
- **Climb the ladder.** Single control-plane before HA; requests-based scheduling before load-aware; metrics before logs.
- **One cluster, logical isolation.** Namespaces, taints, quotas, NetworkPolicies — not more hardware.
- **Security baseline is non-negotiable.** Named sudo user, Ed25519 keys, locked-down SSH, default-deny firewall, encrypted secrets, drained reboots.
- **Mind the SSD.** 256 GB is the tightest resource — lean images, capped retention, `/var` on its own LVM volume.
- **Contain the blast radius.** `--dangerously-skip-permissions` only inside sandbox pods; spend caps on agent API keys.

Build plan

Do phases in order; each ends in a working state. Full detail lives in `homelab-roadmap.md`.

Phase 0 — Prep

Everything staged.

- Plan node names + static IPs
- BIOS: USB boot, VT-x, AC-recovery on
- Collect keys/accounts
- Create git repo + cloud-init autoinstall

Done: autoinstall USB + empty repo ready

Phase 1 — Base OS

Three clean nodes.

- Unattended Ubuntu install on all three
- Key-only SSH from laptop

Done: ssh to all nodes, no passwords

Phase 1.5 — Ansible

Never configure a node by hand.

- Inventory + `ansible all -m ping`
- Baseline + K8s prereqs playbooks
- ansible-vault for secrets

Done: one playbook → identical, hardened nodes

Phase 2 — kubeadm

A real cluster.

- containerd, swap off, sysctls
- `kubeadm init` (1 CP) + join 2 workers
- Calico CNI; k9s on laptop
- Later: 3 CP + kube-vip (HA)

Done: all nodes Ready

Phase 2.5 — Batteries

What kubeadm omits.

- MetalLB, ingress-nginx
- local-path storage

Done: app+PVC reachable via ingress

Phase 2.75 — Hostname access

Reach apps by name over HTTPS.

- Local DNS wildcard → ingress IP
- cert-manager (TLS); ExternalDNS optional
- Later: Tailscale Split DNS

Done: https://app.lab.example works

Phase 3 — Hygiene

Backups, GitOps, secrets.

- etcd snapshot + rehearse restore
- Argo CD + app-of-apps repo
- SOPS/Sealed Secrets; store API keys
- Cert-renewal reminder (~1 yr)

Done: snapshot + Argo + encrypted secrets

Phase 3.5 — Monitoring

See the cluster.

- kube-prometheus-stack (retention capped)
- metrics-server; `kubectl top`

Done: Grafana live; test alert fires

Phase 4 — Rails image

"My project, ready."

- Dockerfile: Ruby+Node+git+agents
- Push to registry (TLS+auth)
- Test as a one-off pod

Done: Rails + agents run in the image

Phase 5 — Agent Sandbox

Pre-warmed envs.

- Install controller + CRDs
- SandboxTemplate `rails-dev`
- SandboxWarmPool (N=2)
- Test claim → hot pod → recycle

Done: claim yields a warm pod instantly

Phase 6 — herdr

The cockpit.

- Install herdr; one pane per agent
- exec into sandbox pods; run claude/codex

Done: 2 agents in 2 pods, one view

Phase 7 — Tailscale

Reach it anywhere.

- Tailscale + subnet router (home node)
- Keep pod traffic on native LAN

Done: drive cluster from outside home

Ongoing / later

After the base is solid.

- Hosted apps via GitOps
- Descheduler → Trimaran (load-aware)
- Loki (logs); Longhorn (if disks added)
- HA control plane; vCluster

Optional futures

Repository structure

```
homelab/
├─ autoinstall/        # cloud-init user-data (identical OS installs)
├─ ansible/
│  ├─ inventory.ini
│  ├─ site.yml
│  └─ roles/           # base, hardening, containerd, kubeadm, ...
├─ clusters/
│  └─ home/
│     ├─ bootstrap/    # Argo CD install
│     ├─ platform/     # MetalLB, ingress, cert-manager, DNS, monitoring,
│     └─ apps/         # your hosted apps
├─ images/
│  └─ rails-dev/       # Dockerfile for the sandbox
└─ docs/               # these planning docs
```

Day-2 operating guidelines

- **Node reboots/upgrades:** always `kubectl drain` → reboot → `kubectl uncordon`. Never let all nodes auto-reboot together.
- **Add a node:** one line in Ansible inventory → re-run `site.yml` → `kubeadm join`.
- **Backups:** scheduled etcd snapshots; back up PVs that matter; rehearse a restore.
- **Certs:** renew kubeadm control-plane certs before the ~1-year expiry.

- **Secrets:** rotate API keys; keep spend caps on them.
- **Capacity:** watch Grafana for pressure and `df -h` for SSD; keep images/retention lean.
- **Change flow:** edit git → Argo CD syncs. Avoid manual `kubectl apply` for anything Argo manages.

Risks & mitigations

RISK	MITIGATION
SSD fills up (images, warm pool, metrics)	`/var` on own LVM volume; lean images; cap retention; monitor `df -h`; add SSD if needed
kubeadm maintenance burden for a beginner	Start single-CP; automate with Ansible; k3s is a legit fallback
Agent Sandbox is young (API churn)	Fallback = plain Deployment of N dev pods as a manual warm pool
Runaway agent starves the cluster	Resource requests/limits + quotas; taint a node for agents
Secrets leak / token burn	SOPS/Sealed Secrets; spend caps; skip-permissions only inside pods
Losing HA on node failure	Move to 3-CP + kube-vip once comfortable; etcd snapshots
Learning overload	Follow the ladder; one phase at a time; each rung is usable

Beginner glossary

Node	One physical machine in the cluster.
Pod	Smallest deployable unit; one or more containers.
Namespace	Logical partition for grouping/limiting workloads.
Deployment	Keeps N identical pods running.
Service / Ingress / LoadBalancer	How traffic reaches pods (MetalLB gives real LAN IPs on bare metal).
CNI	The pod network plugin (Calico).
etcd	The cluster's database (quorum needs an odd number of control-plane nodes).
requests / limits	What a pod reserves vs its hard ceiling (the scheduler places by *requests*).
taint / toleration	Keep pods off a node unless they explicitly tolerate it.
GitOps / Argo CD	Cluster state lives in git; the controller makes reality match it.
SandboxTemplate / WarmPool / Claim	Agent Sandbox: env definition / pool of pre-booted pods / request for one.

Companion documents: `PROJECT-PLAN.md`, `linux-k8s-foundations.md`, `homelab-roadmap.md`, `homelab-decision-doc.md`, plus the diagram files `whole-project-diagram.png` and `architecture-diagram.png`.
Keep this HTML file in the same folder as the two SVGs so the diagrams render.